# 05 — PostgreSQL Core: Data Types, Constraints & Schema Features

## Overview

This module covers the foundational building blocks of PostgreSQL — the data types, constraints, sequences, and schema features that you use in every database you design. Mastering this material means you will always choose the right type, enforce the right rules, and structure your database objects correctly.

## Files in This Module

| File | Topic | Key Concepts |
|---|---|---|
| `01_numeric_types.md` | Integer, decimal, float types | Storage sizes, overflow, SERIAL vs IDENTITY |
| `02_character_types.md` | CHAR, VARCHAR, TEXT | TOAST, collation, string functions |
| `03_date_time_types.md` | DATE, TIME, TIMESTAMP, INTERVAL | Timezone handling, arithmetic |
| `04_boolean_uuid_types.md` | BOOLEAN, UUID | Three-valued logic, UUID versions |
| `05_json_jsonb.md` | JSON and JSONB | Operators, GIN indexes, performance |
| `06_array_types.md` | Native arrays | Containment, GIN, unnest |
| `07_special_types.md` | Ranges, network, full-text, XML | GIST indexes, tsquery, EXCLUDE |
| `08_constraints.md` | All constraint types | NOT NULL, PK, FK, CHECK, EXCLUSION |
| `09_sequences_identity.md` | Sequences and identity columns | SERIAL vs IDENTITY, gaps, caching |
| `10_schemas_namespacing.md` | Schema organization | search_path, permissions, multi-tenant |

## Type Selection Quick Reference

```
Question: What type should I use?

For integers:
  Auto-incrementing PK            → BIGINT GENERATED ALWAYS AS IDENTITY
  Small bounded counter (< 32K)   → SMALLINT
  General integer                 → INTEGER
  Large counter (> 2B rows)       → BIGINT
  Distributed/merged ID           → UUID (v7 preferred)

For decimals:
  Money / financial               → NUMERIC(19, 4)  NEVER float!
  Scientific measurement          → DOUBLE PRECISION
  Percentage                      → NUMERIC(5, 2) CHECK (x BETWEEN 0 AND 100)
```

```
  Coordinates (lat/long)          → DOUBLE PRECISION

For text:
  Any string without strict limit → TEXT
  Enforced max length             → VARCHAR(n)
  Fixed-length codes (country)    → CHAR(n)

For dates/times:
  Calendar date only              → DATE
  Timestamp (always!)             → TIMESTAMPTZ
  Duration                        → INTERVAL

For complex data:
  Tags, keywords, phone numbers   → TEXT[]  (with GIN index)
  Variable attributes             → JSONB   (with GIN index)
  IP addresses                    → INET
  MAC addresses                   → MACADDR
  Date/time ranges                → DATERANGE / TSTZRANGE
  Full-text search                → TSVECTOR (generated column)
```

## Constraint Quick Reference

```
Constraint type     When to use
──────────────────────────────────────────────────────────
NOT NULL            Every column that must have a value
PRIMARY KEY         Exactly one per table (use BIGINT IDENTITY or UUID)
UNIQUE              Business keys: email, SKU, username
FOREIGN KEY         Every reference between tables
CHECK               Domain validation: price > 0, status IN (...)
EXCLUSION           Overlap prevention: no double-booking (GIST)
```

## Learning Path

### Beginner (complete in order)

1. `01_numeric_types.md` — foundation for every table you create
2. `02_character_types.md` — text storage demystified
3. `03_date_time_types.md` — timezone handling correctly from day one
4. `08_constraints.md` — enforce data integrity at the database level

### Intermediate

5. `04_boolean_uuid_types.md` — modern ID strategies
6. `05_json_jsonb.md` — flexible storage when schemas vary
7. `06_array_types.md` — collection patterns without junction tables
8. `09_sequences_identity.md` — auto-increment deep dive

### Advanced

9. `07_special_types.md` — ranges, FTS, geometric, network
10. `10_schemas_namespacing.md` — large application organization

## Common Patterns from This Module

### The canonical table template

```
CREATE TABLE entities (
    entity_id   BIGINT          GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    -- business key (if it exists):
    code        VARCHAR(50)     NOT NULL UNIQUE CHECK (code ~ '^[A-Z0-9\-]+$'),
    -- description:
    name        TEXT            NOT NULL CHECK (length(trim(name)) >= 2),
    -- status:
    is_active   BOOLEAN         NOT NULL DEFAULT TRUE,
    -- metadata:
    created_at  TIMESTAMPTZ     NOT NULL DEFAULT now(),
    updated_at  TIMESTAMPTZ     NOT NULL DEFAULT now()
);
```

### The canonical timestamp pattern

```
-- Always use TIMESTAMPTZ, never TIMESTAMP
-- Always use now() as default, never 'now'::timestamp
created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
```

### The money pattern

```
-- Always NUMERIC, never FLOAT, never the money type
price       NUMERIC(12, 4) NOT NULL CHECK (price >= 0),
total       NUMERIC(19, 4) NOT NULL  -- for summed/computed amounts
```

## Prerequisites

- Basic SQL (SELECT, INSERT, UPDATE, DELETE)
- Understanding of relational databases

## Next Modules

- `../06_Database_Design/` — apply these types in real schema designs
- `../07_Indexes/` — indexing the types covered here
- `../08_Query_Optimization/` — how type choices affect query performance